This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

A memetic algorithm for community detection in signed networks

Shiwei Che[1], Wu Yang[1], and Wei Wang[1]

[1]Information Security Research Center, Harbin Engineering University, Heilongjiang Province, China

Corresponding author: Wu Yang (e-mail: yangwu@hrbeu.edu.cn).

This work was supported by the National Natural Science Foundation of China (Grant No. 61572459 and 61672180).

ABSTRACT Community discovery (i.e. community detection) in signed networks is a division of nodes, such that the edges in the communities are positive and the edges between the communities are negative. Davis and Harary have solved the problem of community detection when a signed graph is balanced or weakly balanced. When the signed network is unbalanced, community detection becomes very complex. In this paper, we propose a novel memetic algorithm (MA) called MACD-SN for community partition (i.e. community detection) in signed networks. Firstly, we present a novel initialization algorithm used in initialization of MACD-SN. This method can accelerate the convergence rate of MACD-SN algorithm. Next, in addition to using frequently-used variation operation (in this paper, variation and mutation are interchangeable), this paper presents a novel crossover operation and a novel variation operation, which contributes to increasing the correctness of the MACD-SN algorithm's operation result and reduces its running time. Lastly, this paper proposes a new local search algorithm, which may enable the algorithm's result to jump away the local best result with a certain probability and draw near the global best result quickly. For testing the performance of MACD-SN algorithm, we have done many experiments using five kinds of synthetic signed networks and five real-world signed networks. The test outcomes show that the proposed algorithm is valid and efficient for signed network cluster partition (i.e. community detection).

INDEX TERMS genetic algorithm**,** social network, signed network, community detection

I. INTRODUCTION

Modern network science is an active field in understanding complex systems. Actually, a lot of complicated systems in various fields can be expressed by means of networks, for example, complex collaborative relationships [1], social systems [2], information systems [3], etc. In these networks, nodes (or vertices) represent individual participants, and edges (or links) represent relationships between participants. A great deal of research efforts have been done on complex networks, such as correlation clustering, dynamic network evolution. Generally speaking, identifying community partition is an important task in complex network analysis. Community structures exist in a lot of network systems, such as politics, economics, engineering, computer science, biology and so on. A comment on network community discovery can be found in Ref. [4]. The purpose of community discovery is to identify clusters with dense links within clusters and only sparse links between clusters [5]. In theoretical research and practical activities, community discovery is of great significance. For instance, in the purchasing relations network between customers and online retailers' products (such as www.taobao.com), identifying clusters of customers with similar interests can establish an effective recommendation system [4, 6].

In human society, many relationships between people are signed, either positive or negative. Compared with traditional networks, the positive and negative edges of signed networks can more accurately describe cooperation (friendship/trust) relations and competition (hostility/distrust) relations. When two people have a relationship of trust, respect or love, the relationship can be regarded as a positive connection. But, the relationship with mistrust, disrespect or hatred can be considered as a negative connection. This network is called signed network [7], that is, the edge weight is greater than 0, indicating a positive relationship; the edge weight is less than 0, indicating a negative relationship; and the edge weight is equal to 0, indicating that there is no relationship between these two individuals. Figure 1 shows a simple signed network. In the figure, the solid line edge represents a positive relationship, and the dotted line edge represents a

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

negative relationship. Community discovery in signed networks is quite different from that in unsigned networks (that is, networks only contain positive connections). In unsigned networks, the community structure is defined as a group of nodes or vertices which have dense connections within groups and sparse connections between groups. Whereas for signed social networks, communities (i.e. clusters) are defined not only by the density of connections but also by the signs of connections. The connections should be densely positive and sparsely negative in a community while densely negative and sparsely positive between communities. Many negative connections exist in the communities and many positive connections exist between the communities make community detection more difficult. A strongly (or weakly) balanced signed network can be divided into two (or more) clusters, so that all connections within clusters are positive, and all connections between clusters are negative [8-10]. However, due to the existence of negative connections in clusters and positive connections between clusters, the real world signed networks are often unbalanced. Therefore, it is a great challenge to design an effective and efficient algorithm to discover the community structure in signed graphs.

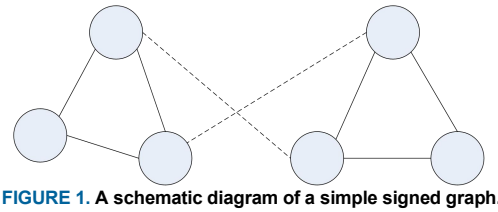


FIGURE 1. A schematic diagram of a simple signed graph.

Unlike previous work, this paper proposes a new memetic algorithm to detect community structure in signed networks. In order to accelerate the convergence of algorithm (decrease the numbers of loop), a novel population initialization method of memetic algorithm is presented. Besides employing the frequently-used variation operation, a novel crossover operation (called randomized two-way crossover operation in this article) and a novel variation operation (called community variation operation in this article) are also proposed, which are capable of enhancing the accuracy of the result of the algorithm and speeding up the convergence speed of population. Randomized two-way crossover operation can preferable retain the hereditary properties of previous generation individuals, and community variation operation is capable of enhancing greatly the chromosomes set (i.e. chromosomes population) multiformity of MACD-SN algorithm. Furthermore, this paper also presents a local solution space search subroutine to drive the optimal result of the offspring individuals of the MACD-SN method approach the global optimal result more quickly in the search region. This subroutine is capable of driving the MACD-SN jump away local best solution and attain global best solution with a specified odds. Many experimental results show that the proposed algorithm is effective and efficient for signed network community partition.

The remaining sections of this thesis are arranged as follows: The work related to this study is illuminated in Section II. A few key concepts and background knowledge connected with this study are introduced in Section III. Section IV describes presented MACD-SN algorithm for community identification (i.e. community detection) in the signed network in detail. This section introduces the proposed MACD-SN algorithm's chromosome coding method, initialization algorithm of chromosomes set, computational formula of fitness used, tournament selection operator for chromosomes selection, crossover operator and mutation operators of genetic operation, local search function, etc. Section V shows the test results on synthetic and real signed networks. Section VI summarizes the whole paper.

II. RELATED WORKS

In recent decades, due to the emergence of a large number of community partition problems, scholars have proposed many algorithms to settle the community partition problems. Girvan and Newman presented a dividing method, which is called GN algorithm. In addition, Newman also put forward a method called FN based on GN algorithm, which uses modularity function. It is a kind of agglomeration algorithm [11]. In the FN algorithm, each node in the graph is initially located in a community with only one node. Afterwards, at every stage, the method continuously consolidates cluster pair with the largest modularity function increment. According to majorization of modularity function, Moore *et al.* [12] proposed a method named CNM to detect community structures in complex networks. In comparison to the FN method, CNM method can save computing time and is appropriate for discovering cluster partition in large scale graphs. In [13], Newman put forward a spectral method as well. The algorithm used a modularity matrix.

In recent years, researchers have proposed many multi-objective majorization algorithms to solve community discovery problems. The multi-objective majorization algorithm finds the optimal solution of the task to be solved by majorizing multiple majorization functions at the same time. These majorization functions evaluate the discovered cluster structure from multiple viewpoints. In the literatures, several frequently cited multi-objective majorization algorithms for solving community discovery problems are listed as follows. Shi *et al.* [14] put forward a multi objective evolutionary algorithm called MOCD. In [15], the authors presented an algorithm named MOEA/D-Net, which is also a multi-objective evolutionary algorithm (MOEA). Liu *et al.* [69] proposed a COMpression based Multi-Objective Evolutionary Algorithm with Decomposition (Com-MOEA/D) for community detection. In the prevenient literatures, there are also two other multi-objective evolutionary algorithms, which are called MODTLBO/D [16] and MODBSA/D [17], respectively. The author of [18]

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

developed a multi objective genetic algorithm called MOGA-Net.

Although there are many excellent algorithms that can be used to detect communities in the unsigned network, these algorithms can not be directly applied to the signed network because of the existence of edge signs in the signed network. Hence, a large number of scholars have proposed a lot of community detection algorithms for signed networks after considering the edge signs of signed networks.

Albayrak *et al.* [19] apply the spectral algorithm to the signed network, and present a spectral method on the basis of the signed Laplacian. They concluded that using signed Laplacian kernel to divide signed networks into two clusters is similar to the ratio cut in unsigned networks. Based on the spectral Laplacian, Tewari *et al.* [20] propose a community detection algorithm for signed networks, and defines the social imbalance (MOIs) on the basis of the l-cycles in signed networks. However, the spectral method is very time consuming.

Arenas *et al.* [21] modified the modularity definition of the unsigned network and extended it to the signed network. Bruggeman *et al.* [22] modified an existing Potts model to include negative links, resulting in a method similar to signed graphs clustering (i.e. community detection). The author of [23] proposed a statistical probability model to identify the community partition of signed networks. Moura *et al.* [24] put forward a mixed integer programming model for clustering problems related to structural balance. Ismail *et al.* [25] presented a high-efficiency two stage algorithm to identify the community structures in signed social networks. The objective functions they used are to minimize frustration and maximize modularity. Dhillon *et al.* [26] presented a scalable and efficient clustering algorithm using balance normalized cut and a multilevel clustering algorithm. Pizzuti *et al.* [27] obtained the community structure of the signed network by maximizing the cluster modularity and minimizing the number of negative edges within communities and the number of positive edges between communities. Liu *et al.* [28] presented two novel evolutionary algorithms and conducted a large number of experiments to compare them. The experimental results show the effectiveness and efficiency of the two algorithms.

Cheung *et al.* [29] presented a random walk algorithm for community detection of signed networks. Firstly, select a node (i.e. vertex) in the network that has not yet assigned a community label, and then use the node as the starting node to perform a specific number of random walks to determine the set of nodes it can reach along the edge path. Next, the authors propose a function to determine which vertices are in the same community as the starting vertex. At the same time, the function considers the distribution of positive connections and negative connections between the community and the rest of the network. From [29], it can see that the result matrix has block characteristics. By using the above function to segment the matrix, the matrix will be split into different block matrices representing different communities.

Du *et al.* [30] proposed a multi objective discrete particle swarm optimization algorithm for multi-resolution signed network clustering. Firstly, the algorithm generates initial population information including location, speed, individual optimal solution and neighborhood information. Next, the new velocity and position of each particle are calculated, and a small disturbance is added to the new position; then, the disturbed position is evaluated, and the neighborhood information and optimal solution of the particle are updated according to the evaluation result.

Jiang *et al.* [31] propose a multi objective evolutionary algorithm, which is based on similarity. In order to consider the sign characteristic of signed networks, the authors proposed two new indexes. One index is the signed similarity index according to the existing similarity index [32]. Another index is the signed tightness index based on the existing tightness index [32]. During the run of the algorithm, when the movement can increase the signed tightness of the cluster (i.e., community), the node will be moved to another cluster. However, if the movement can not increase the signed tightness of the cluster, the node will be independent as a new cluster. A special case is that multiple clusters have the same vertices during the run of the algorithm, so when two clusters have more than half of the same vertices, they will be merged into a new cluster. The author of [33] used the Signed Stochastic Block-Model in the process of community detection. Haseyama *et al.* [34] partition network video by constructing a weighted signed network and maximizing the local modularity.

Recently, some scholars have done some very good works for the community detection of signed networks. For example, Attea *et al.* [48] proposed a new multi-objective signed community detection model and a new anti-frustration heuristic operator for the community detection of signed networks. Ma *et al.* [68] used the relationship between balancedness and spectrum space, and proposed a spectrum algorithm based on leading eigenvectors of signed networks to partition clusters, so as to maximize the balancedness. Zhu *et al.* [49] proposed a new evolution algorithm for community detection in imbalanced signed networks which can be modeled as an optimal partition problem. And the evolving mechanism of nodes is updated by its neighbors' information which leads to form optimal community structure. Yan *et al.* [70] proposed a new modularized convex nonnegative matrix factorization (NMF) model, which combined signed modularized information with convex NMF model to improve the accuracy of community detection in signed and unsigned networks. As for model selection, Yan *et al.* extended the modularity density to signed networks and employed the signed modularity density to determine the number of communities automatically.

Most of the algorithms described in this section are based on global information to identify communities. So far, some

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

scholars have begun to use local information for community detection. The knowledge of using local information for community detection is beyond the scope of this paper, which is not discussed here.

III. FUNDAMENTAL NOTIONS AND BACKGROUND KNOWLEDGE

A. THE DEFINITION OF SIGNED NETWORK AND THE DEFINITION OF CLUSTER DETECTION OF SIGNED NETWORK

A signed social network can be modeled as a graph $G = (V, E)$, where $V = (v_1, v_2, \ldots, v_n)$ is the set of nodes (or vertices), $E = \{(v_i, v_j) \mid v_i, v_j \in V \cap i \neq j\}$ is the set of edges (or links). We can represent graph G by an adjacency matrix $A=(A_{ij})_{n\times n}$, where $A_{ij} = 1$(or -1) if we observe it is the positive (negative) relationship between v_i and v_j, and $A_{ij} = 0$ means that there is no edge between v_i and v_j. If $A_{ij} = 1$, nodes v_i and v_j are positive neighbors of each other; if $A_{ij} = -1$, nodes v_i and v_j are negative neighbors of each other. Given a node $v_i \in V$, $a_i^+ = \sum_{(v_i,v_j)\in E \wedge A_{i,j}=1} A_{ij}$ and $a_i^- = \sum_{(v_i,v_j)\in E \wedge A_{i,j}=-1} |A_{ij}|$ are defined respectively as the positive degree and the negative degree of v_i. $a^+ = \sum_{v_i \in V} a_i^+$ and $a^- = \sum_{v_i \in V} a_i^-$ are the total positive degree and the total negative degree of the signed network, respectively. If for any i and j, $A_{ij} >= 0$, then G is an unsigned network. Here, we do not consider the direction of the edge between any two nodes, that is, G is an undirected graph in this paper.

Let $C = \{c_1, c_2, \ldots, c_k\}$ be a set of communities in G, that is, $c_i \subset V$ for $i = 1, 2, \ldots, k$. The problem of community detection in signed networks is accurately expressed in (1).

$$\begin{cases} A_{ij} > 0, & (v_i,v_j) \in E \wedge v_i \in c_p \wedge v_j \in c_p \\ A_{ij} < 0, & (v_i,v_j) \in E \wedge v_i \in c_p \wedge v_j \in c_q \end{cases} \quad (1)$$

where $p \neq q$, $p, q = 1, 2, \ldots, k$. The problem can be described as identifying the community partition that maximizes the sum of positive edges within communities and negative edges between communities.

If all the positive edges in the signed network are in the communities, and all the negative edges are between the communities, the signed network is balanced; otherwise, the signed network is unbalanced.

B. A SIGNED MODULARITY FUNCTION OF SIGNED GRAPHS

In this section, a signed modularity function Q_s [35] of the signed networks will be described. We will use it later in this study. Its expression is as follows:

$$Q_s = \frac{1}{2a^+ + 2a^-} \sum_i \sum_j \left[A_{ij} - \left(\frac{a_i^+ a_j^+}{2a^+} - \frac{a_i^- a_j^-}{2a^-}\right)\right] \delta(c_i,c_j) \quad (2)$$

where A_{ij} is the element of adjacency matrix A, and c_i,c_j denote the communities to which nodes v_i and v_j belong, respectively. If $c_i = c_j$, $\delta(c_i,c_j) = 1$, otherwise, $\delta(c_i,c_j) = 0$. To understand the meaning of $a_i^+(a_i^-)$, $a_j^+(a_j^-)$ and $a^+(a^-)$, please refer to section III-A.

IV. PROPOSED ALGORITHM

Here, we put forward a memetic algorithm use in community discovery in signed networks, called MACD-SN. At first, we describe the representation of chromosomes in a population. Then, a novel population initialization algorithm is presented, which can significantly speed up the convergence rate of the MACD-SN algorithm. Next, a computational formula for assessing chromosomes within a population is introduced. At the same time, a selection operator for selecting parent chromosomes for subsequent genetic operations is described. Afterwards, we elaborate on a crossover operation (called randomized two-way crossover operation) and two mutation operations (called traditional mutation operation and community mutation operation respectively), which are used in MACD-SN algorithm. The novel crossover operation and novel community mutation operation proposed by us can significantly improve the accuracy of MACD-SN algorithm's result and reduce the running time of the algorithm. In the end, we present a local solution space search subroutine. This subroutine can make the result of MACD-SN algorithm jump out of the local optimal result with a certain probability and approach the global optimal result quickly. The rest of Section IV will elaborate on each theme. Figure 2 shows the flow diagram of MACD-SN algorithm.

A. CHROMOSOME REPRESENTATION

In the classical memetic algorithm, each candidate community division of network is denoted by a chromosome, also called a solution or an individual. A set of a certain number of chromosomes is known as a population, i.e., population $popu=\{ch_1, ch_2, \ldots, ch_q\}$, where ch_j is the jth chromosome within the chromosomes set and q is the number of chromosomes in the set. Chromosome coding methods frequently used in the literatures consist of locus-based coding method and string-based coding method. In order to obtain the corresponding community partition by decoding chromosome conveniently, we make use of string-based coding method. The jth chromosome in the population of memetic algorithm can be represented as: $ch_j=[ge_1,ge_2,\ldots,ge_n]$. Among them, ge_k is the kth gene (or component) of individual ch_j, and the amount of vertices in the signed network is n. The value range of each gene is $\{1, 2,\ldots, n\}$. Genes denote the vertices in the signed network, and

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

the value of the kth gene denote the community to which vertex v_k belongs. In this coding method, if vertices v_i and v_j are in the same community, $ge_i = ge_j$.

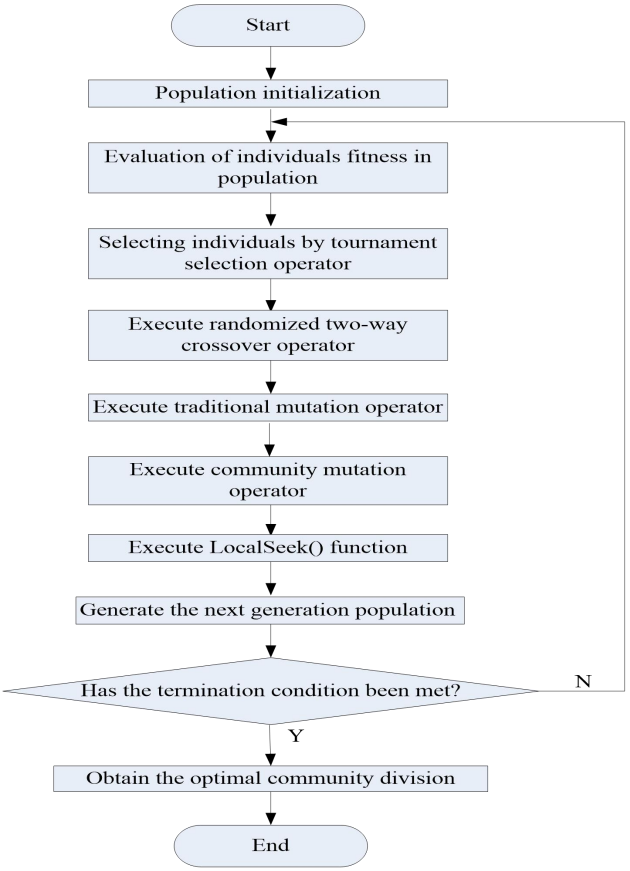


FIGURE 2. The flow diagram of MACD-SN Algorithm.

The chromosome shown in Figure 3(b) is a string-based coding method of the signed network illustrated in Figure 3(a). This signed graph consists of seven vertices. The number of vertices in the network is 1, 2, 3, 4, 5, 6 and 7 respectively. In Figure 3(b), we can see that the gene values assigned to vertices 1, 2, 4 and 5 are all 1, and the gene values assigned to vertices 3, 6 and 7 are all 2. This shows that the network contains two clusters, in which vertices 1, 2, 4 and 5 are in the same cluster, while vertices 3, 6 and 7 are in the other cluster. Figure 3(c) illustrates the cluster partition decoded by the chromosome in Figure 3(b).

B. CREATION OF INITIAL CHROMOSOMES POPULATION

In order to get an excellent memetic algorithm, it is very important to generate a good initial population. The reason is that the properties of the initial chromosomes will have an effect on the convergence rate and the quality of the final result of the method. Therefore, we propose an efficient population initialization algorithm to generate a good initial population and to reduce the convergence time of the whole algorithm. Before introducing the initialization algorithm, we first introduced a definition to be used in the algorithm.

Definition 1. (*Node Imbalance Degree* (*NID*))**.** Let v_i be a node in the signed network G, and c be a community in G, then the node imbalance degree $NID(v_i,c)$ of node v_i relative to community c is calculated as follows:

$$NID(v_i,c)=\sum_{v_j \in c \wedge (v_i,v_j)\in E \wedge A_{ij}=-1} |A_{ij}| + \sum_{v_j \notin c \wedge (v_i,v_j)\in E \wedge A_{ij}=1} A_{ij} \quad (3)$$

Let c_1 and c_2 be two communities in G. Obviously, according to the definition of community structure of signed network, when $NID(v_i, c_1) < NID(v_i, c_2)$, the priority should be given to assigning node v_i to community c_1. Therefore, in the initialization process of the MACD-SN algorithm proposed by us, node k (equivalent to v_i above) is assigned to the neighbor community which reduces its node imbalance degree (NID) to the minimum. Algorithm 1 describes the pseudocode of the initialization process of the presented MACD-SN algorithm.

Algorithm 1. Pseudocode of population creation function initialize() of MACD-SN method.

Algorithm Parameters: chromosomes set size *popu_size*;
Algorithm Input: A matrix *G* representing a signed graph;
Algorithm Output: The initial set of chromosomes generated;

```
for j=1 to popu_size do
    for k=1 to n do
        popu[j][k]= A random integer in the range of 1 to n generated randomly; //popu[] is an array //of chromosomes population. popu[j][k] //represents the kth gene of the jth //chromosome.
    end for
    flag=1;
    while flag do
        flag=0;
        for k=1 to n do
            comms= The set of communities to which the neighbor nodes of node k belong;
            t=∞;
            m=popu[j][k];
            for each community comm ∈ comms do
                if NID(k, comm)<t then
                    t=NID(k, comm);
                    m=comm;
                end if
            end for
            if m≠ popu[j][k] then
                popu[j][k]=m;
                flag=1;
            end if
```


This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

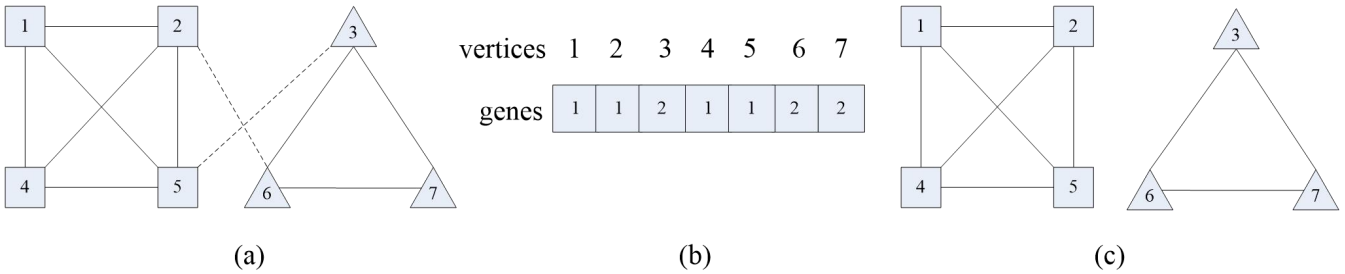


FIGURE 3. **(a)A signed network contains 7 vertices, the solid line edge represents a positive relationship, and the dotted line edge represents a negative relationship; (b)string-based coding method of a individual; (c)schematic diagram of the community structure of the network in (a) decoded by the individual in (b).**

22: end for
23: end while
24: end for
25: return *popu*[];

In the above algorithm, the for loop at lines 1 to 24 is used in producing the initial chromosomes set. The codes in lines 2 to 4 generate an initial chromosome, each gene value of which is a random integer randomly generated in the range of 1 to n. The while loop of lines 6 to 23 continuously optimizes the generated chromosomes, until in a while loop, each node cannot optimize its community label according to its neighbor's community label. The for loop of lines 8 to 22 optimizes chromosome by a single pass loop from 1 to n. In line 9, the community labels of the neighbor nodes of the current node k are stored in the set *comms*. The codes in lines 10 to 17 is responsible for finding the community in *comms* that minimizes the node imbalance degree (NID) of node k, and storing it in m. The codes in lines 18 to 21 check whether the current community label of vertex k is m, if not, use the m value as the community label of vertex k, and set the value of the variable *flag* to 1 to continue the while loop. In the end, the codes in line 25 return the generated population of chromosomes.

Each time the for loop of line 1 is executed, it needs to iterate the *popu_size* times. Each time the for loop of line 2 is executed, it needs to iterate n (n is the number of nodes in the signed network) times. A large number of experiments show that every time the while loop of line 6 is executed, it needs to iterate 8 times on average, no more than 13 times at most. Each time the for loop of line 8 is executed, it needs to iterate n times. It is assumed that the average degree of nodes in the network is d ($d< < n$). The time complexity of the statement in line 9 is $O(d)$. In the worst case, the for loop of line 12 needs to iterate n-1 times for each execution. Therefore, the time complexity of the initialization process of MACD-SN algorithm is $O(popu_size \times n^2)$.

C. FITNESS FUNCTION

In the MACD-SN algorithm proposed in this paper, we use the signed modularity formula Q_s (Eq. 2) introduced in section III-B as the fitness function. According to [35], the larger the value of Q_s, the better the community partition obtained.

D. SELECTION OPERATOR

For selecting parent individuals for subsequent genetic operations, we should propose a good selection operator. In recent years, many methods have been developed as the selection algorithm of memetic algorithm (MA). Tournament selection algorithm is one of them. In order to make the low fitness individuals appear in the offspring chromosomes set, we need a method to control elitism. The tournament selection algorithm meets this requirement, so we chose it. One of the most attractive features of this algorithm is that chromosomes in the current population have the same probability of becoming the parent chromosomes of subsequent genetic operations. The flow chart of the algorithm is shown in Figure 4.

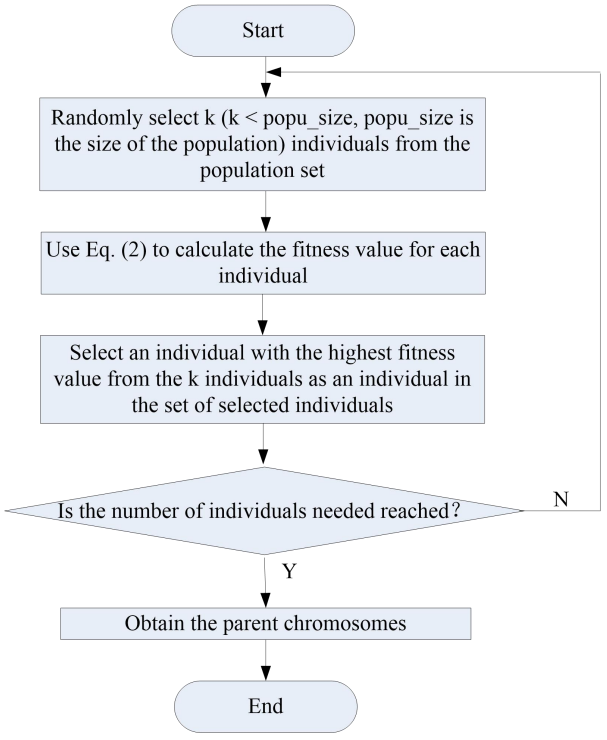


FIGURE 4. **The flow chart of tournament selection algorithm.**

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

The time complexity of tournament selection algorithm is $O(k^2)$ (k is the number of parent chromosomes to be selected by the tournament selection algorithm.).

E. CROSSOVER OPERATOR

Crossover operator is also one of the genetic operators of MA algorithm. The crossover operator applies simultaneously to two parent chromosomes chosen by the selection operator, and reproduces new chromosomes through interchanging contents between the selected chromosomes. Hence, the chromosomes produced through crossover operator possess simultaneously the genetic properties of two chromosomes of the previous generation [36]. The execution process of crossover operator is: (1)choosing two chromosomes from the parent chromosomes set; (2)exchanging contents between them in the light of crossover operation rules to generate novel offspring chromosomes; (3)giving two novel chromosomes. First, randomly select a crossing location. Afterwards, according to the specified crossover probability, the corresponding contents of the two parent chromosomes around the location are interchanged. Traditional crossover operations are listed below: the uniform crossover, the one-point crossover, the one-way crossover, the two-point crossover, the two-way crossover, and so on.

For taking full advantage of the community partition information of two individuals of the previous generation, enhance the accuracy of the offsprings produced through crossover operation and reduce the running time of algorithm, this paper presents a novel crossover operation, which is named randomized two-way crossover operation. The pseudo code of randomized two-way crossover operation is shown in algorithm 2.

Algorithm 2. Pseudocode for function Crossover() that performs randomized two-way crossover operation.

Algorithm Parameters: Two parent individuals (i.e. chromosomes) ch_1 and ch_2 selected by tournament selection algorithm;
Algorithm Input: A matrix G representing a signed graph;
Algorithm Output: The two generated individuals;

```
for i=1 to 2 do
    for j=1 to n do
        ch1_s[j]='U';  //ch1_s[j] = 'U' means ch1[j] has
          //not been touched yet, ch1_s[j] = 'V' means
          //ch1[j] has been touched.
        ch2_s[j]='U'; //ch2_s[j] = 'U' means ch2[j] has
          //not been touched yet, ch2_s[j] = 'V' means
          //ch2[j] has been touched.
        di[j]=0;
    end for
    ct=1;
    repeat
        Generate a random number r between 0 and 1;
        if 0 ≤ r ≤ 0.5 then
            Randomly choose an untouched component in
              ch1, assuming that the chosen component is
              ch1[k], that is, ch1_s[k]='U';
            Locate all the component locations in ch1 that
              have the same component value as the
              ch1[k], assuming that these components are
              ch1[k1], ch1[k2],..., ch1[ks];
            if di[k1] ≠ 0 ∧ di[k2] ≠ 0 ∧ ⋯ ∧ di[ks] ≠0
                then
                for each u∈{k1, k2, ... , ks} do
                    ch1_s[u]='V';
                end for
                goto 11;
            end if
            for each u∈{k1, k2, ... , ks} do
                if di[u]==0 then
                    di[u]=ct;
                end if
                ch1_s[u]='V';
            end for
        else
            Randomly choose an untouched component in
              ch2, assuming that the chosen component is
              ch2[k], that is, ch2_ s[k]='U';
            Locate all the component locations in ch2 that
              have the same component value as the
              ch2[k], assuming that these components are
              ch2[k1], ch2[k2],..., ch2[ks];
            if di[k1] ≠ 0 ∧ di[k2] ≠ 0 ∧ ⋯ ∧ di[ks] ≠0
                then
                for each u∈{k1, k2, ... , ks} do
                    ch2_s[u]='V';
                end for
                goto 26;
            end if
            for each u∈{k1, k2, ... , ks} do
                if di[u]==0 then
                    di[u]=ct;
                end if
                ch2_s[u]='V';
            end for
        end if
        ct=ct+1;
    until all component locations of the descendant
          individual di are written in;
end for
return the generated descendant individuals d1 and d2;
```

Figure 5 shows an executive process of the crossover operator of the MACD-SN method. Figure 6 shows an example of community partition encoded by chromosomes ch_1, ch_2, d_1 and d_2 in Figure 5, respectively.

In Figure 5, ch_1 and ch_2 are two parent individuals chosen by the tournament selection algorithm from the previous generation population. The steps of obtaining a descendant individual d_1 by the individuals ch_1 and ch_2 of the previous

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

generation are listed below. Step one: Label all components of ch_1 and ch_2 as untouched, assign 0 to all components of the descendant individual d_1. Step two: Generate a random number *r*. Suppose *r* = 0.7. Because 0.5<*r* ⩽ 1, in individual ch_2, an untouched component 4 is randomly chosen, and all components with the identical component value as component 4 in individual ch_2, namely components 2, 4, 6 and 9, are located. Afterwards, 1 is assigned to the counter *ct*, and the present *ct* value is written in the components 2, 4, 6 and 9 of the descendant individual d_1. Next, components 2, 4, 6 and 9 within individual ch_2 are labeled as touched. Step three: Generate a random number *r*. Suppose *r* = 0.4. Because 0 ⩽ *r* ⩽ 0.5, in individual ch_1, an untouched component 8 is randomly chosen, and all components with the identical component value as component 8 in individual ch_1, namely components 8 and 9, are located. Afterwards, *ct* + 1 = 2 is assigned to *ct*, and write the current *ct* value into the component 8 of the descendant individual d_1. Due to the prevenient operation of the algorithm has written component 9 of d_1, this algorithm step doesn't change it. Component 9 of the individual d_1 remains unchanged. Afterwards, components 8 and 9 in individual ch_1 are labeled as touched. Step four: Generate a random number *r*. Suppose *r* = 0.2. Because 0 ⩽ *r* ⩽ 0.5, in individual ch_1, an untouched component 2 is randomly chosen, and all components with the identical component value as component 2 in individual ch_1, namely components 2 and 6, are located. Because the components 2 and 6 of d_1 have been written in, the algorithm first labels components 2 and 6 of ch_1 as touched, and then randomly chooses another untouched component 5 in ch_1. Next, the algorithm locates all components with the identical component value as component 5 in individual ch_1, namely components 4 and 5. Afterwards, let *ct* = *ct*+1=3, and write the current *ct* value into the component 5 of the descendant individual d_1. Due to the prevenient operation of the algorithm has written component 4 of d_1, this operation doesn't change it. Then, components 4 and 5 in individual ch_1 are labeled as touched. Step five: Generate a random number *r*. Suppose *r* = 0.6. Because 0.5<*r* ⩽ 1, in individual ch_2, an untouched component 7 is randomly chosen, and all components with the identical component value as component 7 in individual ch_2, namely components 1, 3 and 7, are located. Afterwards, let *ct* = *ct*+1=4, and write the current *ct* value into the components 1, 3 and 7 of the descendant individual d_1. Then, components 1, 3 and 7 in individual ch_2 are labeled as touched. At this time, the individual d_1 has been filled, so the work of producing descendant individual d_1 has been completed. We obtain descendant individual d_1. The generative steps of the other descendant individual d_2 are analogous to that of d_1. Figure 5 (c) shows the generation process of descendant individual d_2.

If the chromosome contains *m* communities on average, the time complexity of crossover operation is $O(m \times n)$. In the worst case, there are *n* communities in the parent

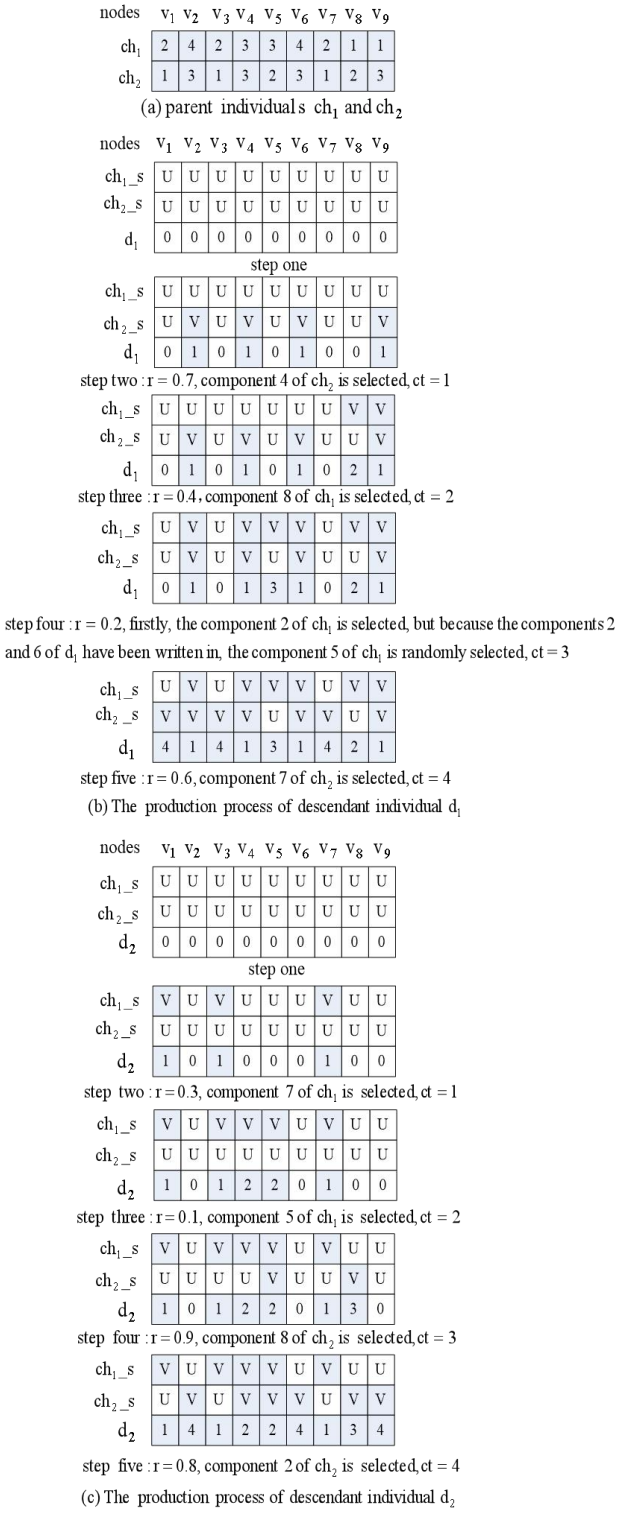


FIGURE 5. An executive process of the crossover operator of the MACD-SN method. In the figure, U represents that the relevant component has been untouched, and V represents that the relevant component has been touched. The figure in the rectangle represents the label of the cluster that contains the relevant node. The value 0 represents that the relevant component has not been written. (a)Previous generation individuals ch_1 and ch_2. (b)The producing process of individual d_1. (c)The producing process of individual d_2.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

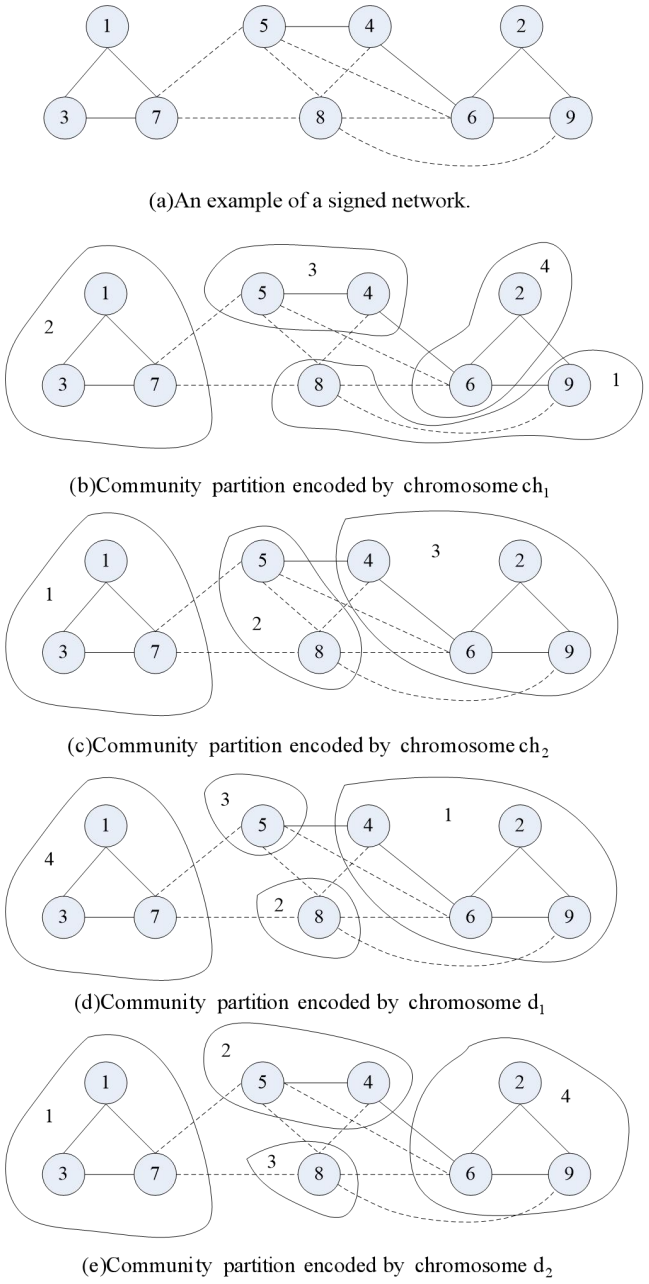


FIGURE 6. An example of community partition encoded by chromosomes ch_1, ch_2, d_1 and d_2 in Figure 5, respectively. A solid line indicates a positive edge and a dotted line indicates a negative edge.

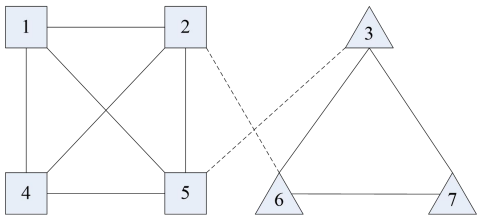
chromosomes of crossover operation, then the time complexity of crossover operation is $O(n^2)$.

F. MUTATION OPERATOR

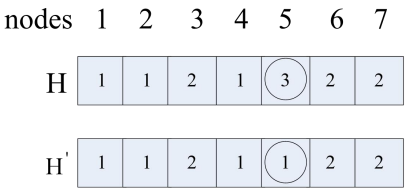
In the MACD-SN method, in addition to the frequently-used variation operation (this paper calls it the traditional variation operation), we also put forward the other variation operation (this paper calls it the community variation operation). In this paper, mutation and variation are interchangeable.

As indicated in Figure 3, in classical string-based coding method, every component denotes a node, and the value of component stands for the cluster containing the node. In the cluster identification methods, the frequently-used mutation operation first stochastically chooses a vertex v_s, after that stochastically chooses a neighbour vertex v_t of v_s (v_s and v_t are not included in the identical cluster), and at last assigns the cluster label (component value) of v_t to the component corresponding to vertex v_s [37]. The time complexity of frequently-used mutation operation is $O(1)$.

Figure 7 shows an instance of using a traditional variation operation. As indicated in Figure 7, a chromosome H is selected first, and afterwards a vertex 5 on chromosome H is randomly chosen. The adjacent vertices of vertex 5 include vertices 1, 2, 3 and 4. Suppose that vertex 2 is randomly chosen. Thus, the component value of vertex 2 is assigned to the component corresponding to vertex 5.



(a)An example signed network. The solid line edge represents a positive relationship, and the dotted line edge represents a negative relationship.



(b) An example of a traditional mutation operation on a signed network shown in (a).

FIGURE 7. A schematic diagram of a traditional mutation operator.

Previous to describe community mutation operator in detail, the definition of community imbalance degree is first described.

Definition 2. (*Community Imbalance Degree* (*CID*))**.** Let *comm* be a community in the signed network *G*, then the community imbalance degree *CID* (*comm*) of community *comm* is calculated as follows:

$$CID\ (comm)=\frac{1}{n_C}\sum_{v_i \in comm} NID(v_i,comm) \quad (4)$$

where $\boldsymbol{n_C}$ is the number of nodes in community *comm*. The smaller the value of number of *CID* (*comm*), the more evident the cluster structure of cluster *comm*.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

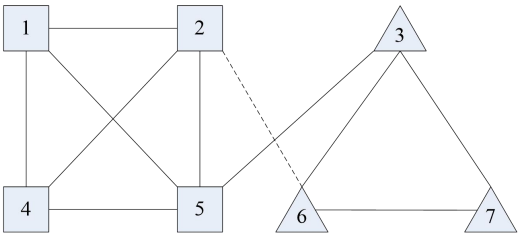
The community variation operation will be described in detail below. If the *CID* (*comm*) is higher than a parameter $\boldsymbol{\delta}$, we think that the vertices in *comm* can not constitute a signed cluster. At this point, we need to use community mutation operator to reallocate the vertices in *comm* to more suitable clusters. In algorithm 3, the operation steps of community mutation operator of the MACD-SN method are presented. Figure 8 is an explanatory drawing of the execution steps of community mutation operator of the MACD-SN method.

Algorithm 3. Pseudocode for function CommunityMutation() that performs community mutation operator.

Algorithm Parameters: parent individual *H*, a threshold parameter δ;
Algorithm Input: A matrix *G* representing a signed graph;
Algorithm Output: improved individual H';
1: *comms*= The set of clusters in *H*;
2: k=$-\infty$;
3: m=0;
4: for each cluster *comm*∈*comms* do
5: if *CID*(*comm*)>k then //*CID* (*comm*) is the //community imbalance degree of community //*comm*.
6: k=*CID*(*comm*);
7: m=*comm*;
8: end if
9: end for
10:if $m\neq 0 \wedge CID(m)>\delta$ then
11: *ns* = All vertices in m;
12: $H' = H$;
13: while every vertex $v_s \in ns$ do
14: Stochastically choose a positive neighbour v_t of vertex v_s (v_s and v_t are not included in the identical cluster);
15: if there is v_t satisfying the condition then
16: $H'[v_s]=H'[v_t]$;
17: end if
18: end while
19: qs_H=Q *s* value of *H* computed using formula (2);
20: $qs_{H'}$=Q *s* value of H' computed using formula (2);
21: if $qs_{H'}>qs_H$ then
22: return H';
23: end if
24: end if
25: return *H*;

Figure 8(a) shows an example of a simple signed graph. Figure 8(b) illustrates an individual produced through the graph in Figure 8(a). This individual is composed of three clusters, i.e. cluster 1, 2 and 3. By equation (4), we may get *CID* (*comm1*) = $\frac{4}{2}$ = 2, *CID* (*comm2*) = $\frac{5}{2}$ = 2.5, and *CID* (*comm3*) = 1/3. Let $\boldsymbol{\delta}$ = 2.1. Thus, cluster 2 is chosen for mutation. Among the neighbors of vertex $\boldsymbol{v_2}$, vertices $\boldsymbol{v_1}$ and $\boldsymbol{v_4}$ are its positive neighbors, and they are not in the same cluster as vertex $\boldsymbol{v_2}$. Since the cluster labels of vertices $\boldsymbol{v_1}$ and $\boldsymbol{v_4}$ are both 1, the cluster label of vertex $\boldsymbol{v_2}$ are set as 1. Among the positive neighbors of vertex $\boldsymbol{v_5}$, vertices $\boldsymbol{v_1}$, $\boldsymbol{v_3}$ and $\boldsymbol{v_4}$ are not in the same community as vertex $\boldsymbol{v_5}$. Suppose vertex $\boldsymbol{v_4}$ is selected randomly. Therefore, the community label 1 of vertex $\boldsymbol{v_4}$ is assigned to vertex $\boldsymbol{v_5}$.

The time complexity of community mutation operation is $O(n)$.



(a)An example signed network.

nodes	v_1	v_2	v_3	v_4	v_5	v_6	v_7
H	1	2	3	1	2	3	3

(b) Individual H before mutation. $CID(comm1)=\frac{4}{2}=2$, $CID(comm2)=\frac{5}{2}=2.5$, and $CID(comm3)=\frac{1}{3}$. $\delta=2.1$, so community 2 was chosen to mutate.

nodes	v_1	v_2	v_3	v_4	v_5	v_6	v_7
H'	1	1	3	1	1	3	3

(c) The mutated individuaal H'.

FIGURE 8. An instance of the execution process of community variation operation of the MACD-SN method.

G. LOCAL SEARCH FUNCTION

For memetic algorithm, the local search process is a crucial part. A good local search subroutine may significantly increase the accuracy of the ultimate result of the algorithm and reduce the time to find the optimal solution. Therefore, this paper presents a subroutine called LocalSeek(), which achieves the above goals well. Algorithm 4 describes the details of the subroutine LocalSeek().

Algorithm 4. LocalSeek().

Algorithm Parameters: best individual *bestOffspring* in the offspring individuals set, a threshold parameter ρ;
Algorithm Input: A matrix *G* representing a signed graph;
Algorithm Output: revised individual *bestOffspring*;

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

1: A stochastic permutation of the natural numbers in the range of 1 to *n* is generated and saved in a set called *seq*;
2: for *count*=1 to *n* do
3: *i*=*seq*[*count*];
4: *comms*=∅;
5: Put the clusters (i.e. communities) containing most positive neighbors of vertex *i* into the set *comms*(there may be multiple clusters that meet the condition);
6: *k*= ∞;
7: *m*=0;
8: if *comms*≠ ∅ then
9: for each cluster *comm* ∈ *comms* do
10: *newComm*= Put node *i* into cluster *comm* to form a new cluster;
11: if *CID* (*newComm*)<*k* then //*CID* (*newComm*) //is the community imbalance //degree of community *newComm*.
12: *k*=*CID* (*newComm*);
13: *m*=*comm*;
14: end if
15: end for
16: else
17: Put the clusters containing negative neighbors of vertex *i* into the set *comms*;
18: for each cluster *comm* ∈ *comms* do
19: *newComm*= Put node *i* into cluster *comm* to form a new cluster;
20: if *CID*(*newComm*)<*k* then //*CID* (*newComm*) //is the community imbalance //degree of community *newComm*.
21: *k*=*CID*(*newComm*);
22: *m*=*comm*;
23: end if
24: end for
25: end if
26: if *m*≠0 then
27: *newDivision*=Put the vertex *i* of *bestOffspring* into cluster *m* to produce a novel individual;
28: qs_o= Q_s value of individual *bestOffspring* was computed through formula (2);
29: qs_n= Q_s value of individual *newDivision* was computed through formula (2);
30: if $qs_n > qs_o$ then
31: *bestOffspring* =*newDivision*;
32: else if random number *r* between 0-1 generated randomly< ρ
33: *bestOffspring* =*newDivision*;
34: end if
35: end if
36:end for
37:return *bestOffspring*;

In algorithm 4, the codes in line 1 generate a random permutation of the natural numbers from 1 through *n* and put it in a set called *seq*. The iteration from 1 to *n* of lines 2 to 36 attempt to assign each node in the network to another more appropriate cluster (i.e. community) in the order of the nodes specified in the set *seq*. Obviously, according to the definition of the signed community, we should first try to put the current node *i* into the community to which most of its positive neighbors belong. When node *i* has no positive neighbors, we should try to put node *i* into the community that one of its negative neighbors belongs to. Algorithm 4 embodies this idea. The codes in line 5 put the clusters containing the most positive neighbours of the current vertex *i* into the set *comms*. (there may be multiple clusters that meet the condition). The codes in lines 9 to 15 deal with the case where node *i* has positive neighbors. Suppose there are *p* clusters in *comms*, which are $comm_1, comm_2, \dots, comm_p$. After node *i* is assigned to clusters $comm_1, comm_2, \dots, comm_p$, the new clusters formed are $newComm_1$, $newComm_2$, ... , $newComm_p$, respectively. The codes in lines 9 to 15 are responsible for finding the community $comm_j$ so that the *CID* ($newComm_j$) is the minimum value in *CID* ($newComm_1$), *CID* ($newComm_2$), ... , *CID* ($newComm_p$), that is, $comm_j = \arg \min_{comm_r \in comms} CID(comm_r \cup i)$. Obviously, at this point, $comm_j$ is the most suitable community in the set *comms* to merge with node *i*. The codes in lines 17 to 24 handle the case where node *i* has only negative neighbors. The codes in line 17 put the clusters containing the negative neighbours of vertex *i* into the set *comms*. Suppose there are *q* clusters in *comms*, which are $comm_1, comm_2, \dots, comm_q$. After node *i* is assigned to clusters $comm_1, comm_2, \dots, comm_q$, the new clusters formed are $newComm_1$, $newComm_2$, ... , $newComm_q$, respectively. The codes in lines 18 to 24 are responsible for finding the cluster $comm_j$ so that the *CID* ($newComm_j$) is the minimum value in *CID* ($newComm_1$), *CID* ($newComm_2$), ... , *CID* ($newComm_q$), that is, $comm_j = \arg \min_{comm_r \in comms} CID(comm_r \cup i)$. Obviously, at this point, $comm_j$ is the most suitable cluster in the set *comms* to merge with node *i*. The codes in lines 26 to 35 are responsible for deciding whether node *i* can be placed in the cluster $comm_j$ (i.e. cluster *m*) found. If possible, place node *i* in the cluster *m*. The codes in line 27 put the node *i* of the individual *bestOffspring* into the cluster *m* to form a new individual *newDivision*. The codes in line 28 use equation (2) to calculate the Q_s value for individual *bestOffspring* and assign the calculated Q_s value to variable qs_o. The codes in line 29 use equation (2) to calculate the Q_s value for individual *newDivision* and assign the calculated Q_s value to variable qs_n. The codes in line 30 determines if qs_n is greater than qs_o. If qs_n is greater than qs_o, the codes in line 31 take the individual *newDivision* as the best individual *bestOffspring* in the offspring population. If qs_n

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

is not greater than qs_o, the codes in line 32 randomly generate a real number r between 0 and 1, and determines whether it is less than the input parameter ρ of the algorithm. If less than, the codes in line 33 take the individual *newDivision* as the best individual *bestOffspring* in the offspring population. The codes in line 37 return the improved individual *bestOffspring*.

Our subroutine LocalSeek() has one outstanding merit. This merit is that it may accept a worse result within a certain odds. In the process of solving, this merit enables LocalSeek() to increase the accuracy of the result of the algorithm, and also helps the algorithm jump away from the local best solution. In algorithm 4, the conditional statement in line 32 implements this function. When the algorithms find the local optimal solution, some other algorithms will terminate the iteration. On account of some other algorithms can not find the global best solution by movement in a small area near the local best solution. Nevertheless, the LocalSeek() presented in this paper can do this with a specified odds, namely, it may adopt a result with a certain odds that is worse than the present result. This is helpful for the subroutine to jump out of the local best result and attain the global best result after a few moving operations.

In algorithm 4, the time complexity of the statement in line 1 is $O(n \times \log n)$. Each time the for loop of line 2 is executed, it needs to iterate n times. It is assumed that the average degree of nodes in the network is d ($d << n$). The time complexity of the statement in line 5 is $O(d)$. In the worst case, the for loop of line 9 needs to iterate n-1 times for each execution. Similarly, in the worst case, the for loop of line 18 needs to iterate n-1 times for each execution. The time complexity of the statement in line 17 is $O(d)$. Because $d << n$, the time complexity of LocalSeek() is O (n^2).

Algorithm 5 gives the pseudo code of the main function of MACD-SN algorithm.

Algorithm 5. MACD-SN method.

Algorithm Parameters: population size *popu_size*, number of parent individuals selected by tournament selection algorithm k, crossover probability p_1, mutation probability p_2 of traditional mutation operator, mutation probability p_3 of community mutation operator, a threshold parameter δ used in community mutation operator, a threshold parameter ρ used in the function LocalSeek(), amount of iterations without revision *gt*;

Algorithm Input: A matrix G representing a signed graph;

Algorithm Output: a cluster partition *comms* of the signed graph;

```
1:  popu= initialize(popu_size); //produce initial individual
                 //population;
2:  repeat
3:     Use formula (2) to compute the fitness function
          value of every individual in the population popu;
4:     for i=1 to k do //Using tournament selection
          //algorithm to select parent individuals for
          //subsequent genetic operations.
5:        Randomly select k individuals from the
          population set popu;
6:        Using the selected k individuals to build a
          maximum heap;
7:        offs[i]= The top element of the maximum heap;
             //offs[] is a individuals set selected for
             //subsequent genetic operations.
8:     end for
9:     i=1;
10:    while i≤k-1 do
11:       if rand(0,1)≤ p1 then
12:          (offs[i], offs[ i+1])=Crossover(offs[i], offs[i+1]);
                //Perform a randomized two-way crossover
                //operation. The Crossover function returns
                //two individuals, offs[i] and offs[i + 1].
13:       end if
14:       i=i+2;
15:    end while
16:    for i=1 to k do
17:       if rand(0,1)≤ p2 then
18:          offs[i]=The traditional mutation operator is
                executed on the offs[i];
19:       end if
20:    end for
21:    for i=1 to k do
22:       if rand(0,1)≤ p3 then
23:          offs[i]=CommunityMutation(offs[i], δ); //The
                //community mutation operator is executed
                //on the offs[i];
24:       end if
25:    end for
26:    bestOffspring= The individual with the maximum
          fitness function value in offs;
27:    bestOffspring=LocalSeek(bestOffspring, ρ);
28:    popu=popu + offs + bestOffspring;
29:    popu=The set of the first popu_size individuals
          with the largest fitness in the population popu;
30:until  no improved amount of iterations for the
          optimal individual in population popu > = gt;
31:comms= Cluster partition of individual with the
          biggest fitness function value in the population
          popu;
32:return comms;
```

According to the analysis of the prevenient sections, in algorithm 5, the time complexity of the statement in line 1 is $O(popu_size \times n^2)$. The time complexity of the statement in line 3 is $O(popu_size)$. The time complexity of the codes in lines 4 to 8 is $O(k^2)$ (k is the number of parent chromosomes to be selected by the tournament selection algorithm.) . On average, the time complexity

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

of the codes in lines 10 to 15 is $O(k\times m\times n)$ (m is the average number of communities contained in a single chromosome.) . The time complexity of the codes in lines 16 to 20 is $O(k)$. The time complexity of the codes in lines 21 to 25 is $O(k\times n)$. The time complexity of the statement in line 26 is $O(k)$. The time complexity of the statement in line 27 is $O(n^2)$. The time complexity of the statement in line 29 is $O(popu_size\times \log(popu_size))$. The time complexity of the statement in line 31 is $O(popu_size+n)$. Through a large number of experiments, we can see that the average execution times of repeat statement in line 2 is about $popu_size/3$. Therefore, the time complexity of MACD-SN algorithm is $O(popu_size\times k\times m\times n+popu_size\times n^2)$.

V. EXPERIMENTAL RESULTS

In our experiment, the MACD-SN algorithm proposed in this paper is verified in the synthetic and real-world signed networks. We also make comparisons with other four famous algorithms, namely, FEC[38], SSL[39], DM[40] and SISN [41]. We use normalized mutual information (NMI) [66] to assess the capability of the methods. Given that A and B are two partitions of a network, then $NMI(A, B)$ is defined as follows.

$$NMI(A,\boldsymbol{B})=\frac{-2\sum_{i=1}^{C_A}\sum_{j=1}^{C_B}C_{ij}\,log\left(\frac{C_{ij}N}{C_{i.}C_{.j}}\right)}{\sum_{i=1}^{C_A}C_{i.}\,log\left(\frac{C_{i.}}{N}\right)+\sum_{j=1}^{C_B}C_{.j}\,log\left(\frac{C_{.j}}{N}\right)} \tag{5}$$

where N is the number of nodes of the network, C is a confusion matrix. C_{ij} equals to the number of nodes shared in common by community i in partition A and by community j in partition B. C_A (or C_B) is the number of clusters in partition A (or B), $C_{i.}$ (or $C_{.j}$) is the sum of elements of C in row i (or column j). NMI takes value between 0 and 1. The larger the value of number of NMI, the more evident the cluster structure obtained.

In this paper, the source codes of FEC, SSL, DM and SISN are obtained from the original author. MACD-SN is implemented by C# 4.0 using Microsoft Visual Studio 2010. In the experiment, we set the parameters $popu_size$ = 1000, k=500, $p_1 = p_2 = p_3 = 0.3$, δ=3.9, gt = 3, and ρ= 0.15.

A. SYNTHETIC SIGNED NETWORKS

By using the generation model in reference [38], the synthetic signed networks used in our experiment are produced. This model can be described as

$$SG(c, n, k, pm_{in}, pm\text{-}, pm+)$$

where c is the number of communities, n is the number of vertices in each community, k is the average degree of the vertices, pm_{in} is the probability that a link falls within a community, pm- is the probability that a link within communities is negative, and pm+ is the probability that a link between communities is positive. pm- and pm+ are also known as the yawp parameters.

In order to verify our method effectively, in our experiment, we produce five kinds of synthetic signed networks, which have different characteristics. These kinds of synthetic signed networks include two different types of networks: balanced networks and unbalanced networks. Among them, only networks 1 is balanced, and other kinds of networks are imbalanced.

Networks 1: The generation model of this kind of networks is SG (6, 42, 42, pm_{in}, 0, 0). Among them, in the interval [0, 1], pm_{in} increases gradually, and the increment of each step is 0.1. Because the positive edges are within the clusters and the negative edges lie between the clusters, this kind of signed networks are balanced. We carry out five methods on 11 networks of this kind of networks. In Figure 9, we describe the results of the five algorithms.

As shown in Figure 9, the MACD-SN and SISN can accurately discover all communities in the networks. This means that the two algorithms are not sensitive to the variation of the parameter pm_{in} and they have good detection ability within the balanced signed graphs. The

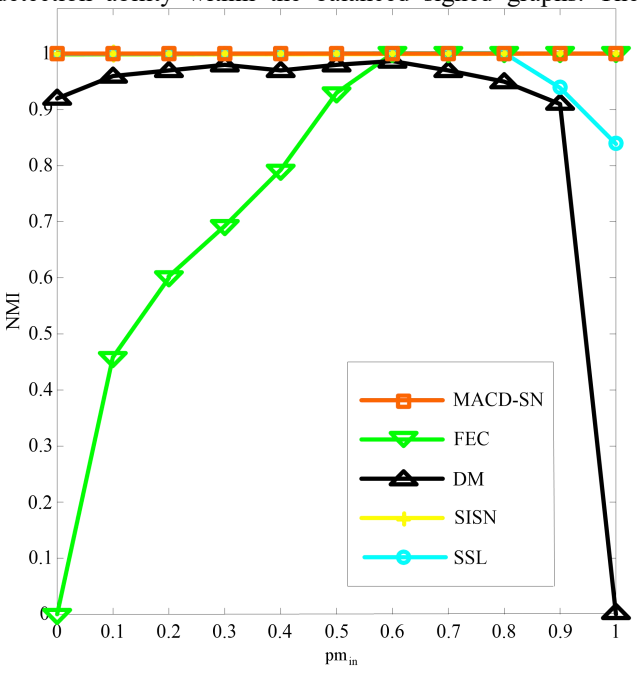


FIGURE 9. The output of MACD-SN algorithm and four comparison algorithms on Networks 1.

SSL algorithm also has excellent detection ability, when $pm_{in}\leqslant 0.8$, it can correctly identify the communities.

Networks 2: The generation model of this kind of networks is SG(6, 42, 42, 0.5, pm-, 0). Among them, in the interval [0, 1], pm- increases gradually, and the increment of each step is 0.1. This kind of signed networks is imbalanced, but the yawp only consists in the clusters. In these signed networks, the positive edges do not exist between clusters. The greater the numerical value of pm-, the more minus edges the clusters contain.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

The outputs of the five methods running on Networks 2 are shown in Figure 10. As we can see, when *pm-* varies from 0 to 1, the numerical values of *NMI* of the MACD-SN algorithm and SSL algorithm are both 1. This result shows that MACD-SN and SSL can perfectly identify the community structures in this kind of signed networks. The performance of the other three algorithms is relatively poor.

Networks 3: The generation model of this kind of networks is SG(6, 42, 42, 0.5, 0, *pm+*). Among them, in the interval [0, 1], *pm+* increases gradually, and the increment of each step is 0.1. In this kind of networks, noises only exists between clusters, and there are no negative edges in the clusters, so this kind of signed networks is imbalanced. The greater the numerical value of *pm+*, the more positive edges there are between clusters in the network.

Figure 11 shows the outputs of the five methods on Networks 3. In Figure 11, we may observe that MACD-SN is capable of identifying the clusters in the networks well, but when *pm+* > 0.1, the detection ability of SSL decreases. On the whole, the sorting results of the five algorithms by performance are MACD-SN, SSL, SISN, DM and FEC.

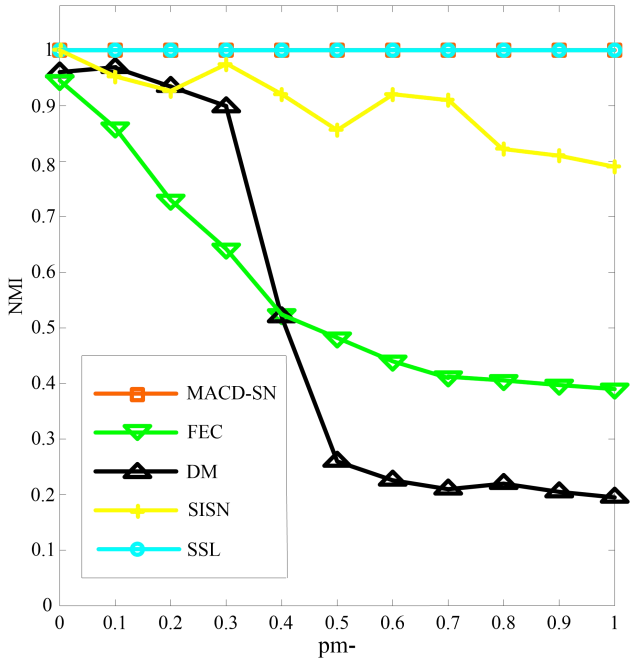


FIGURE 10. The outputs of MACD-SN algorithm and four comparison algorithms on Networks 2.

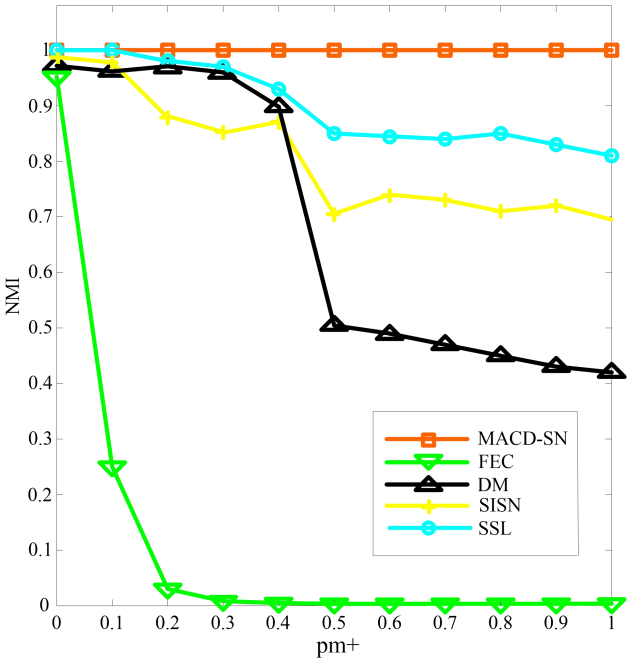


FIGURE 11. The outputs of MACD-SN algorithm and four comparison algorithms on Networks 3.

Networks 4: The generation model of this kind of networks is SG(6, 42, 42, 0.5, *pm-*, 0.5). Among them, in the interval [0, 1], *pm-* increases gradually, and the increment of each step is 0.1. In this kind of networks, not only the negative edges exist within the clusters, but also the positive edges exist between the clusters, so this kind of signed networks is imbalanced. The greater the numerical value of *pm-*, the more minus edges the clusters contain.

The outputs of the five methods running on Networks 4 are shown in Figure 12. We can see that in Figure 12, when *pm-* ≤ 0.3, MACD-SN can perfectly identify communities in the networks; when *pm-* > 0.3, the performance of MACD-SN decreases. The SSL also has good accuracy, but the DM and FEC have very bad accuracy. Among the five algorithms, the performance of SISN is in the third place.

Networks 5: The generation model of this kind of networks is SG(6, 42, 42, 0.5, 0.5, *pm+*). Among them, in the interval [0, 1], *pm+* increases gradually, and the increment of each step is 0.1. Due to some negative edges consist in the clusters and a number of positive edges consist between the clusters, this kind of signed networks is imbalanced. The greater the numerical value of *pm+*, the more positive edges there are between clusters in the network.

Figure 13 shows the results of the five methods running on networks 5. We can see that in Figure 13, MACD-SN has the best accuracy among the five algorithms. SSL is in second place. The rest are in order: SISN, FEC, and DM. In order to further prove the high performance of our presented algorithm, in Chart 1, we show the average values of *NMI*. These average values are derived from the results of every method executing on the above five kinds of signed networks. In Chart 1, we may observe that our algorithm shows the best accuracy among the five comparison algorithms.

B. REAL SIGNED NETWORKS

About the real signed networks, in the experiment, we choose two real-world networks with true cluster partition and three real-world networks without true cluster partition to test our method. Two real signed networks that possess true cluster partition are Gahuku-Gama subtribes network

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

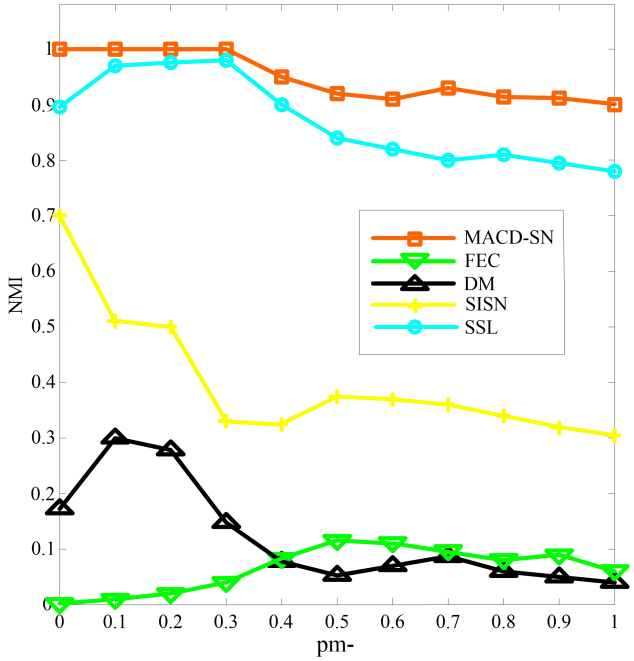


FIGURE 12. The outputs of MACD-SN algorithm and four comparison algorithms on Networks 4.

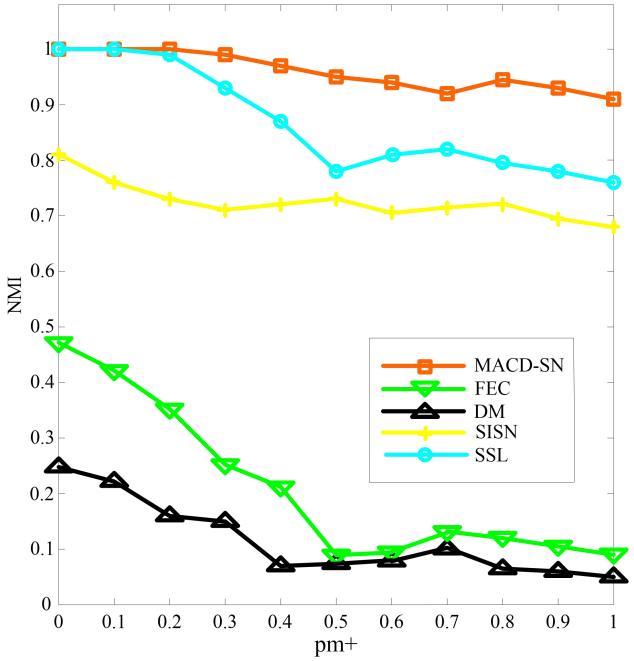


FIGURE 13. The outputs of MACD-SN algorithm and four comparison algorithms on Networks 5.

(GGSN) [43] and Slovene parliamentary party network (SPPN) [42], separately. Three real signed networks that lack true cluster partition are Slashdot network [44], Country network [45], and Epinions network [44].

The SPPN shows the relationships among the Slovenian parliament's 10 political parties in 1994. The plus and minus edges separately represent the alike and unalike relationships between the political parties. The true cluster partition of the SPPN consists of two clusters. The GGSN was created based on Reads study on the cultures of the Eastern Central Highlands of New Guinea. This network describes the political alliance and enmities among the 16 Gahuku-Gama subtribes, which were distributed in a particular area and were engaged in warfare with one another in 1954. The positive and negative links of the network correspond to political arrangements with positive and negative ties, respectively.

The results of our algorithm running on SPPN are shown in Figure 14. In Figure 14, we may observe that MACD-SN detects two clusters on SPPN. The red circle vertices are in one cluster, and the blue square vertices are in the other cluster. Since all the plus edges are in the clusters and all the minus edges are between clusters, SPPN is balanceable. The final results of our algorithm is in line with the true cluster partition of SPPN.

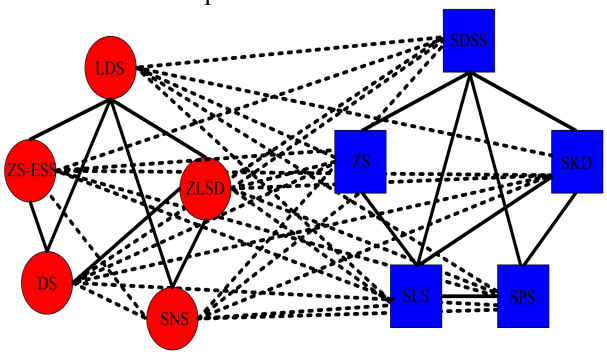


FIGURE 14. Results of the MACD-SN running on SPPN.

The results of MACD-SN algorithm running on GGSN are shown in Figure 15. In Figure 15, the vertices with the identical color and shape are in the same cluster. There are three clusters in GGSN. In Figure 15, we can see that most positive edges are in the clusters, and all negative edges are between the clusters. This shows that GGSN is an imbalanced signed network. The outputs of MACD-SN algorithm is in line with the true cluster partition of the GGSN.

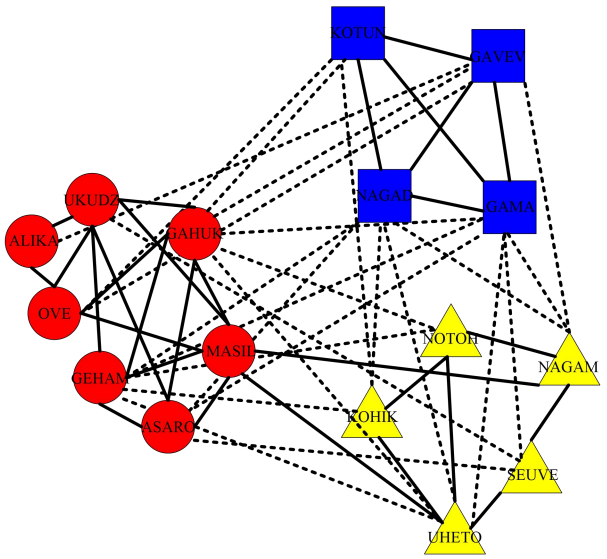


FIGURE 15. Results of the MACD-SN running on GGSN.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

Chart 1. The average NMI values of the five algorithms.

	MACD-SN	SSL	SISN	DM	FEC
Networks I	**1**	0.9805	**1**	0.8746	0.7646
Networks II	**1**	**1**	0.8985	0.5091	0.5323
Networks III	**1**	0.9014	0.806	0.6846	0.114
Networks IV	**0.9494**	0.8699	0.3991	0.1225	0.0637
Networks V	**0.9597**	0.8659	0.7251	0.1185	0.2116

The Country network was generated using the Correlates of War data set from 1996 to 1999 [45]. The vertices represent the nations, the plus edges denote the military leagues, and the minus edges represent the military antagonisms. In the tests, we remove insular vertices in the graph and all connected components except the largest connected components in the graph, and only retain the maximal connected component. This maximal component consists of 144 vertices and 1243 edges.

MACD-SN algorithm puts all the vertices into eight clusters. The outcomes of the algorithm are presented in Figures 16 and 17. Figure 16 visually presents the cluster partition of the Country network. In Figure 16, the minor circles are used to represent the vertices, solid lines are used to represent the positive edges, and dashed lines are used to represent the negative edges. The vertices with the identical color are in the identical cluster and constitute the larger circle. Figure 17 shows an adjacency matrix rearranged based on cluster partition. In Figure 17, pink and blue dots represent positive and negative edges, severally. Eight clusters of vertices are divided by solid lines with a green color. The number on the right in Figure 17 is the number of the clusters, which have a one-to-one correspondence with the number near the big circles in Figure 16.

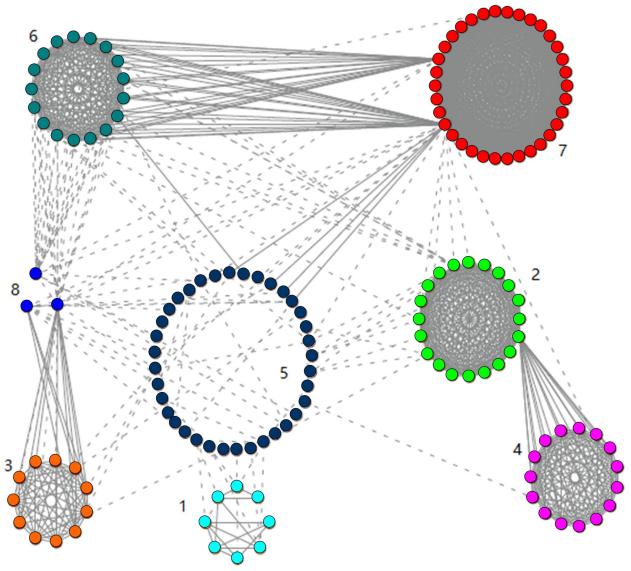


FIGURE 16. Result of the MACD-SN running on Country network.

In Figures 16 and 17, we can see that, (1)Among the eight clusters, clusters 2, 3, 4, 6 and 7 have dense positive edges. (2)Among the eight clusters, there are two clusters with relatively sparse positive edges, which are clusters 1 and 8 respectively. (3)The cluster 5 is composed of many peripheral vertices, in which there are few edges. (4) The negative edges mainly exist between clusters. (5) There are no negative edges between the clusters 2 and 4, but there are a large amount of positive edges between one vertex of the cluster 2 and the vertices in the cluster 4. (6) There are

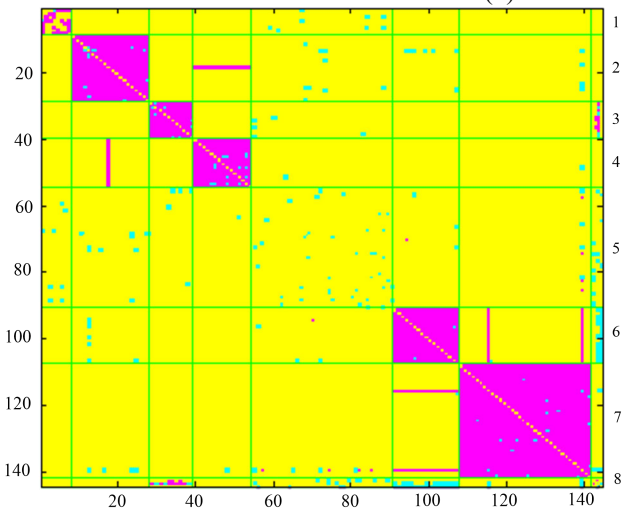


FIGURE 17. Result of the MACD-SN running on Country network.

few edges between clusters 6 and 7, but there are a large amount of positive edges between two vertices of the cluster 7 and the vertices in the cluster 6.

We also compare the result of our algorithm with that of SSL. Figure 18 presents the output of the SSL on the Country signed graph. In Figure 18, we may observe that the running result of MACD-SN is significantly superior to that of SSL.

We also run our algorithm MACD-SN on the Slashdot network and Epinions network [44]. Epicions is a famous network of consumers reviews. Users may believe or not believe other users reviews. Slashdot is a debate website where users can see other users as pals or foes. After all the insular vertices within the network are removed, 126828 vertices are left in the Epinions network and 73099 in the Slashdot network. In our experiment, the two remaining

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

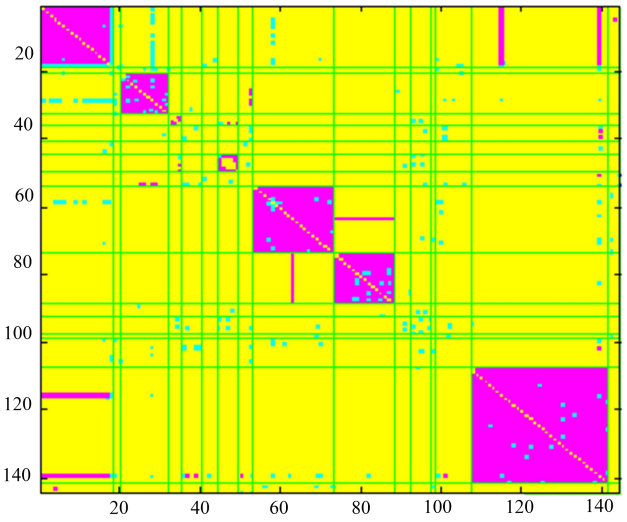


FIGURE 18. Output of SSL running on the Country network.

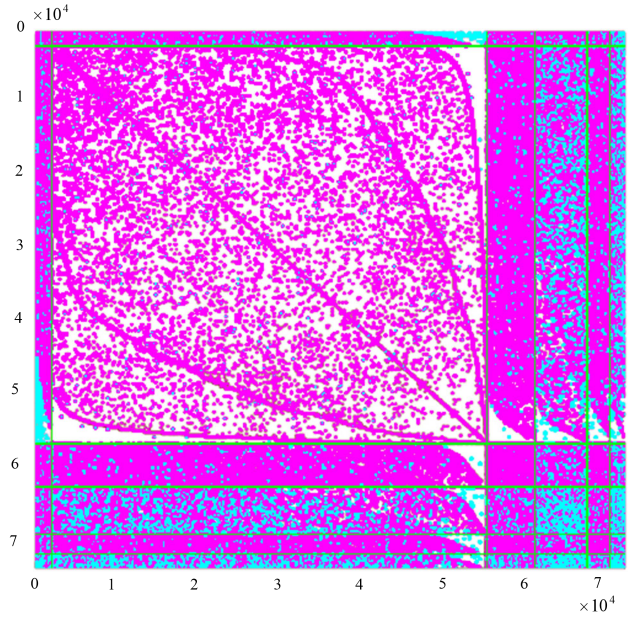


FIGURE 19. Result of the MACD-SN running on Slashdot network.

networks were used. In the Epinions network, our algorithm identifies sixteen clusters. In Slashdot network, our algorithm identifies six clusters. Figure 19 shows an adjacency matrix of the Slashdot network rearranged based on cluster partition. In Figure 19, pink and blue dots represent positive and negative edges, respectively. Six clusters of vertices are divided by solid lines with a green color. According to the order from top left to bottom right on the main diagonal, the number of vertices in each cluster is 1741, 54398, 5922, 6413, 2746 and 1879, severally. In Figure 19, we may observe that the largest cluster with 54398 vertices is loose, and the rest of clusters are dense. Among the six clusters, the clusters with 6413 vertices and 1879 vertices include a large number of negative edges, while the clusters with 1741 vertices, 5922 vertices and 2746 vertices include a large number of positive edges.

VI. CONCLUSIONS

In the research field of signed graphs, cluster structure is an important network feature. For the sake of better study and take advantage of the signed networks, it is crucial to discover their cluster partition. In this paper, we propose a memetic method named MACD-SN for cluster partitions in signed networks. The individual coding method of MACD-SN algorithm adopts the well-known string-based coding method. In order to speed up the convergence, we proposed a new initialization algorithm for the cluster partitions of signed networks. The fitness function of MACD-SN algorithm uses the Q_s function presented in [35]. In order to select parent individuals for succedent genetic operations, we adopt a well-known operator (i.e. tournament selection operator), which provides chromosomes in the parent population identical probabilities to be chosen for subsequent genetic operators. In addition to the frequently-used mutation operation, this paper also presents a novel crossover operation and a novel mutation operation. The novel randomized two-way crossover operation can preferable retain the hereditary properties of the previous generation individuals, and the novel community mutation operator may greatly enhance the population diversity. Moreover, this paper presents a novel local search subroutine, which may enhance the accuracy of the ultimate output of the MACD-SN and reduce its running time, and enable the algorithm to jump out of the local best solution with a specified odds and attain the global best solution. For verifying the detection ability of the proposed algorithm, a large number of tests have been executed on five kinds of synthetic signed graphs and five real signed graphs. Next, we compare the test outcomes with four well-known signed network cluster partition methods. The comparison outcomes show that the performance of MACD-SN method is better than the other four methods, which indicates that the method proposed in this paper is an excellent method to identify cluster partitions in signed networks. The disadvantage of MACD-SN algorithm is that it can't detect overlapping communities in signed networks. We will solve this problem in our future work.

REFERENCES

[1] Z. Xia, Z. Bu, Community detection based on a semantic network, Knowl.-Based Syst. 26 (2012) 30–39.
[2] P. Doreian, A. Mrvar, Partitioning signed social networks, Soc. Networks 31 (2009) 1–11.
[3] M.E.J. Newman, The structure of scientific collaboration networks, Proc. Natl. Acad. Sci. USA, vol. 9781400841356, pp. 221–226, 2011.
[4] S. Fortunato, Community detection in graphs, Phys. Rep. 486 (3) (2010) 75–174.
[5] M. Girvan, M.E.J. Newman, Community structure in social and biological networks, Proc. Natl. Acad. Sci. USA 99 (12) (2002) 7821–7826.
[6] K.P. Reddy, M. Kitsuregawa, P. Sreekanth, S.S. Rao, A graph based approach to extract a neighborhood customer community for

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

collaborative filtering, in: Databases in Networked Information Systems, Springer, Berlin, Heidelberg, 2002, pp. 188–200.

[7] P. Doreian, A. Mrvar, A partitioning approach to structural balance, Social Networks 18 (2) (1996) 149–168.

[8] B. Yang, X. Zhao, and X. Liu, ''Bayesian approach to modeling and detecting communities in signed network,'' in *Proc. AAAI*, Austin, TX, USA, 2015, pp. 1952–1958.

[9] D. Cartwright and F. Harary, ''Structural balance: A generalization of Heider's theory,'' *Psychol. Rev.*, vol. 63, no. 5, pp. 277–293, 1956.

[10] J. A. Davis, ''Clustering and structural balance in graphs,'' *Hum. Relations*, vol. 20, no. 2, pp. 181–187, 1967.

[11] M.E. Newman, Fast algorithm for detecting community structure in networks, Phys. Rev. E 69 (2004) 066133.

[12] A. Clauset, M.E. Newman, C. Moore, Finding community structure in very large networks, Phys. Rev. E 70 (2004) 066111.

[13] M.E.J. Newman, Modularity and community structure in networks, Proceedings of the National Academy of Sciences of the United States of America 103 (2006) 8577–8582.

[14] C. Shi, Z. Yan, Y. Cai, B. Wu, Multi-objective community detection in complex networks, Appl. Soft Comput. 12 (2012) 850–859.

[15] M. Gong, L. Ma, Q. Zhang, L. Jiao, Community detection in networks by using multiobjective evolutionary algorithm with decomposition, Phys. A Stat.Mech. Appl. 391 (15) (2012) 4050–4060.

[16] D. Chen, F. Zou, R. Lu, L. Yu, Z. Li, J. Wang, Multi-objective optimization of community detection using discrete teaching-learning-based optimization with decomposition, Inf. Sci. 369 (2016) 402–418.

[17] F. Zou, D. Chen, S. Li, R. Lu, M. Lin, Community detection in complex networks: multi-objective discrete backtracking search optimization algorithm with decomposition, Appl. Soft Comput. 53 (2017) 285–295.

[18] C. Pizzuti, A multiobjective genetic algorithm to find communities in complex networks, IEEE Trans. Evol. Comput. 16 (2012) 418–430.

[19] Jerome Kunegis, Stephan Schmidt, Andreas Lommatzsch, J urgen Lerner, Ernesto W. DeLuca, and Sahin Albayrak. Spectral analysis of signed graphs for clustering, prediction and visualization. In Proceedings of the SIAM International Conference on Data Mining, 2010,559-570.

[20] Kai-Yang Chiang, Cho-Jui Hsieh, Nagarajan, Inderjit S. Dhillon, Ambuj Tewari. Prediction and Clustering in Signed Networks: A Local to Global Perspective. Journal of Machine Learning Research, 2014, 15: 1177-1213.

[21] S. Gómez, P. Jensen and A. Arenas. Analysis of community structure in networks of correlated data. Phys. Rev. E, 80(1): 016114, 2009.

[22] V.A. Traag, Jeroen Bruggeman, Community detection in networks with positive and negative links, Phys. Rev. E 80 (3) (2009) 036115.

[23] H.W. Shen, Community Structure: An Introduction, Springer, Berlin, Heidelberg, 2013.

[24] R. Figueiredoa, G. Moura, Mixed integer programming formulations for clustering problems related to structural balance, Social Networks 35 (4) (2013) 639–651.

[25] P. Anchuri, M.M. Ismail, Communities and balance in signed networks: A spectral approach, in: 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, IEEE Computer Society, 2012, pp. 235–242.

[26] K.Y. Chiang, J.J. Whang, I.S. Dhillon, Scalable clustering of signed networks using balance normalized cut, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, ACM, 2012, pp. 615–624.

[27] A. Amelio, C. Pizzuti, Community mining in signed networks: a multiobjective approach, in: Proceedings of the 2013 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, ACM, 2013, pp. 95–99.

[28] Y. Li, J. Liu, C. Liu, A comparative analysis of evolutionary and memetic algorithms for community detection from signed networks, Soft Comput. 18 (2) (2014) 329–348.

[29] B. Yang, W. Cheung, and J. Liu, ''Community mining from signed social networks,'' *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 10, pp. 1333–1348, Oct. 2007.

[30] Q. Cai, M. Gong, S. Ruan, Q. Miao, and H. Du, ''Network structural balance based on evolutionary multiobjective optimization: A two-step approach,'' *IEEE Trans. Evol. Comput.*, vol. 19, no. 6, pp. 903–916, Dec. 2015.

[31] C. Liu, J. Liu, and Z. Jiang, ''A multiobjective evolutionary algorithm based on similarity for community detection from signed social networks,'' *IEEE Trans. Cybern.*, vol. 44, no. 12, pp. 2274–2287, Dec. 2014.

[32] J. Huang, H. Sun, Y. Liu, Q. Song, and T. Weninger, ''Towards online multiresolution community detection in large-scale networks,'' *PLoS ONE*, vol. 6, no. 8, p. e23829, Aug. 2011.

[33] J. Q. Jiang, ''Stochastic block model and exploratory analysis in signed networks,'' *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 91, no. 6, p. 062805, Jun. 2015.

[34] R. Harakawa, T. Ogawa, and M. Haseyama, ''Extracting hierarchical structure of web video groups based on sentiment-aware signed network analysis,'' *IEEE Access*, vol. 5, pp. 16963–16973, Aug. 2017.

[35] S. Gómez, P. Jensen, and A. Arenas, "Analysis of community structure in networks of correlated data," *Phys. Rev. E.*, vol. 80, no. 1, p. 016114, 2009.

[36] D. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley, Reading, MA, 1989.

[37] C. Pizzuti, A multiobjective genetic algorithm to find communities in complex networks, IEEE Trans. Evol. Comput. 16 (3) (2012) 418–430.

[38] B. Yang , W. Cheung , J. Liu , Community mining from signed social networks, IEEE Trans. Knowl. Data Eng. 19 (10) (2007) 1333–1348 .

[39] B. Yang , X. Liu , Y. Li , et al. , Stochastic blockmodeling and variational Bayes learning for signed network analysis, IEEE Trans. Knowl. Data Eng. 29 (9) (2017) 2026–2039 .

[40] P. Doreian , A. Mrvar , A partitioning approach to structural balance, Soc. Netw. 18 (2) (1996) 14 9–16 8 .

[41] X. Zhao , B. Yang , X. Liu , H. Chen , Statistical inference for community detection in signed networks, Phys. Rev. E 95 (4) (2017) 042313 .

[42] S. Kropivnik , A. Mrvar , An analysis of the slovene parliamentary parties network, Dev. Stat. Methodol. (1996) 209–216 .

[43] K.E. Read , Cultures of the central highlands, new guinea, Southwest. J. Anthropol. 10 (1) (1954) 1–43 .

[44] J. Leskovec , D. Huttenlocher , J. Kleinberg , Signed networks in social media, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, ACM, 2010, pp. 1361–1370 .

[45] P. Doreian , A. Mrvar , Structural balance and signed international relations, J. Soc. Struct. 16 (2015) 1–49 .

[46] Hua J.,Yu j., and Yang M., "Fast clustering for signed graphs based on random walk gap", *Social Networks,* vol. 60, pp. 113-128, 2020.

[47] Brusco M. and Doreian P., "Partitioning signed networks using relocation heuristics, tabu search, and variable neighborhood search", *Social Networks,* vol. 56, pp. 70-80, 2019.

[48] Attea B., Rada H., Abbas M., and Özdemir, S., "A new evolutionary multi-objective community mining algorithm for signed networks", *Applied Soft Computing Journal,* vol. 85, 2019.

[49] Zhu X., Ma Y., and Liu Z., "A novel evolutionary algorithm on communities detection in signed networks", *Physica A: Statistical Mechanics and its Applications,* vol. 503, pp. 938-946, 2018.

[50] Ping S., Liu D., Yang B., Zhu Y., Chen H., and Wang Z., "Community Detection in Signed Networks Based on the Signed Stochastic Block Model and Exact ICL", *IEEE Access,* vol. 7, 2019.

[51] Chen J.,Liji U., Wang H., and Yan Z., "Community Mining in Signed Networks Based on Dynamic Mechanism", *IEEE Systems Journal*, vol. 13, no. 1, 2019.

[52] Yan C. and Chang Z., "Modularized convex nonnegative matrix factorization for community detection in signed and unsigned networks", *Physica A: Statistical Mechanics and its Applications*, vol. 539, 2020.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2020.3006108, IEEE Access

[53] Liu X., Song W., Musial K., Zhao X., Zuo W. and Yang B., "Semi-supervised stochastic blockmodel for structure analysis of signed networks", *Knowledge-Based Systems,* 2020.

[54] Liu D., Zhang Y., Liang R., Li B., and Xia Z., "Signed Network Community Mining Based on Fine-grained Signed Stochastic Block Model", *2019 2nd International Conference on Artificial Intelligence and Big Data (ICAIBD),* 2019.

[55] Bonchi F., Galimberti E., Gionis A., Ordozgoiti B., and Ruffo G., "Discovering Polarized Communities in Signed Networks", *CIKM '19: Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, Beijing, China, pp. 961-970, 2019.

[56] Hu B., Wang H., Yu X., Yuan W. and He T., "Sparse network embedding for community detection and sign prediction in signed social networks", *Journal of Ambient Intelligence and Humanized Computing,* vol. 10, no. 1, pp. 175-186, 2019.

[57] Chen J., Liu D., Hao F. and Wang H., "Community detection in dynamic signed network: an intimacy evolutionary clustering algorithm", *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, no. 2, pp. 891-900, 2020.

[58] He X., Du H., Du W., and Feldman M., "A Community Structure in Fully Signed Static Networks", *Hsi-An Chiao Tung Ta Hsueh/Journal of Xi'an Jiaotong University,* vol. 52, no. 2, pp. 45-51, 2018.

[59] Wang S., Hu G., Pan Z., Zhang J., and Li D., "A game-theoretic approach for community detection in signed networks", *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences,* vol. E102A, no. 6, pp. 796-807, 2019.

[60] Li Z., Chen J., Fu Y., Hu G., Pan Z., and Zhang L., "Community Detection Based on Regularized Semi-Nonnegative Matrix Tri-Factorization in Signed Networks", *Mobile Networks and Applications,* vol. 23, no. 1, pp. 71-79, 2018.

[61] Zhang Y., Liu Y., Ma X., and Song J., "Community detection in signed networks by relaxing modularity optimization with orthogonal and nonnegative constraints", *Neural Computing and Applications,* 2019.

[62] Hu B., Wang H., and Zheng Y., "Sign prediction and community detection in directed signed networks based on random walk theory", *International Journal of Embedded Systems,* vol. 11, no. 2, pp. 200-209, 2019.

[63] Girdhar N., and Bharadwaj K., "Community Detection in Signed Social Networks Using Multiobjective Genetic Algorithm", *Journal of the Association for Information Science and Technology,* vol 70, no. 8, pp. 788-804, 2019.

[64] Zahedinejad E., Crawford D., Adolphs C., and Oberoi J., "Multiple Global Community Detection in Signed Graphs", 4th Future Technologies Conference, FTC 2019, San Francisco, CA, United states, pp. 688-707, 2019.

[65] Wu Y., Chao P., Ying W., He L., and Chen S., "A Conical Area Evolutionary Algorithm Based on Modularity Q for Community Detection from Signed Networks", *20th IEEE International Conference on Computational Science and Engineering and 15th IEEE/IFIP International Conference on Embedded and Ubiquitous Computing, CSE and EUC 2017,* Guangzhou, Guangdong, China, pp. 57-62, 2017.

[66] Cai M., Shen B., Ma L., Jiao L., "Discrete particle swarm optimization for identifying community structures in signed social networks", *Neural Networks,* vol. 58, pp. 4-13, 2014.

[67] Ruby, Kaur I., "An advanced automated approach for community mining in signed social networks", *2017 International Conference on Energy, Communication, Data Analytics and Soft Computing, ICECDS 2017,* Chennai, India, pp. 665-670, 2017.

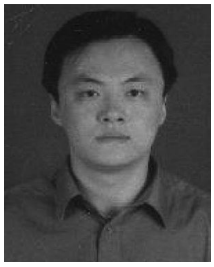
[68] Y. Ma, X. Zhu, Q. Yu, "Clusters detection based leading eigenvector in signed networks", *Physica A: Statistical Mechanics and its Applications,* vol. 523, pp. 1263-1275, 2019.

[69] Z. Liu, Y. Ma, X. Wang, "A Compression-Based Multi-Objective Evolutionary Algorithm for Community Detection in Social Networks", *IEEE Access,* vol. 8, 2020.

[70] C. Yan, Z. Chang, "Modularized convex nonnegative matrix factorization for community detection in signed and unsigned networks*", Physica A: Statistical Mechanics and its Applications,* vol. 539, 2020.



Shiwei Che is currently a Ph.D. candidate in the Department of Computer Science and Technology, Harbin Engineering University. He received his M.E. degree in 2010 from the Department of Computer Science and Technology of Xinjiang University, Xinjiang, China. His main research interests include social networks and community detection.



Wu Yang received a Ph.D. degree in Computer System Architecture Specialty of Computer Science and Technology School from Harbin Institute of Technology. He is currently a professor and doctoral supervisor of Harbin Engineering University. His main research interests include wireless sensor network, peer-to-peer network and information security. He is a member of ACM and senior member of CCF.



Wei Wang received a Ph.D. degree in Computer System Architecture Specialty of Computer Science and Technology School from Harbin Institute of Technology. He is currently an professor in Harbin Engineering University. His main research interests include social networks and community detection.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.